GENETIC ALGORITHM

Code:

```
import random

# ----------------------------------------
# 1. JOB SHOP DATA FROM RESEARCH PAPER
# ----------------------------------------

# Each operation = (Machine Number, Processing Time)

data = {
    "A": {
        "quantity": 20, "due": 1000,
        "plans": [
            [(14,10),(15,61),(12,43),(11,39),(8,47)],
            [(14,10),(15,18),(13,30),(4,32),(11,39),(8,47)]
        ]
    },
    "B": {
        "quantity": 15, "due": 455,
        "plans": [
            [(5,10),(9,10),(1,33),(5,38)],
            [(5,10),(8,6),(4,40),(5,38)],
```

```
        [(6,25),(10,41),(14,24),(6,38),(10,37)],
        [(6,25),(10,41),(13,12),(12,30),(7,39),(10,37)]
    ]
},
"C": {
    "quantity": 18, "due": 345,
    "plans": [
        [(4,29),(2,19),(7,21)],
        [(4,29),(8,29),(6,9),(7,13)],
        [(5,39),(14,33),(2,11),(1,10),(12,16)],
        [(5,39),(1,6),(5,30),(3,39),(2,29),(1,19)],
        [(10,69),(13,33),(8,29)]
    ]
},
"J": {
    "quantity": 22, "due": 265,
    "plans": [
        [(4,33),(15,20)],
        [(1,22),(13,20),(7,35),(9,31),(12,45),(14,17)],
        [(1,22),(12,76),(9,19),(3,48),(14,17)]
    ]
},
"K": {
    "quantity": 11, "due": 335,
    "plans": [
        [(10,27),(1,40)],
        [(2,6),(13,36)],
```

```
            [(6,30),(5,73),(4,31)]
        ]
    },
    "L": {
        "quantity": 14, "due": 670,
        "plans": [
            [(11,29),(15,44),(5,5),(12,41),(11,15)],
            [(11,29),(15,24),(2,42),(11,15)],
            [(3,22),(1,5),(9,39),(13,19)],
            [(3,18),(6,5),(4,18),(7,39),(10,7)],
            [(3,22),(7,10),(12,38),(13,19)]
        ]
    }
}

part_types = list(data.keys())
transport_time = 3

# ----------------------------------------
# 2. INITIALIZE CHROMOSOME
# ----------------------------------------

def create_chromosome():
    chromosome = []

    for part in part_types:
        number = len(data[part]["plans"])
```

```
        chromosome.append(random.randint(1, number))

    return chromosome

# ----------------------------------------
# 3. SIMULATION AND FITNESS EVALUATION
# ----------------------------------------

def evaluate(chromosome):

    jobs = []

    # Create individual jobs based on quantity
    for i, part in enumerate(part_types):

        selected_plan = chromosome[i] - 1
        route = data[part]["plans"][selected_plan]

        for unit in range(data[part]["quantity"]):
            jobs.append({
                "part": part,
                "route": route,
                "due": data[part]["due"],
                "next": 0,
                "ready": 0,
                "finish": 0
            })
```

```
# Machine availability
machines = {m: [] for m in range(1, 16)}

# Events: (completion time, machine, job index)
events = []

current_time = 0

while True:

    # Complete finished operations
    remaining = []

    for finish, machine, job_id in events:

        if finish <= current_time:
            job = jobs[job_id]

            job["next"] += 1
            job["finish"] = finish

            if job["next"] < len(job["route"]):
                job["ready"] = finish + transport_time

        else:
            remaining.append((finish, machine, job_id))
```

```
events = remaining

# Assign available machines using SPT
assigned = False

for machine in range(1, 16):

    if any(event[1] == machine for event in events):
        continue

    candidates = []

    for j, job in enumerate(jobs):

        if job["next"] >= len(job["route"]):
            continue

        if job["ready"] > current_time:
            continue

        next_machine, processing_time = job["route"][job["next"]]

        if next_machine == machine:
            candidates.append((processing_time, j))

    if candidates:
```

```
        # Shortest Processing Time rule
        candidates.sort(key=lambda x: (x[0], x[1]))

        processing_time, job_id = candidates[0]

        finish = current_time + processing_time

        events.append((finish, machine, job_id))
        assigned = True

# Check whether all jobs are completed
if all(job["next"] >= len(job["route"]) for job in jobs):
    break

# Move simulation time to the next event
future_times = [event[0] for event in events]

for job in jobs:
    if job["next"] < len(job["route"]):
        if job["ready"] > current_time:
            future_times.append(job["ready"])

if future_times:
    current_time = min(future_times)
elif not assigned:
    break
```

```
    # Calculate performance measures
    completion_times = [job["finish"] for job in jobs]

    makespan = max(completion_times)

    total_tardiness = sum(
        max(0, job["finish"] - job["due"])
        for job in jobs
    )

    mean_tardiness = total_tardiness / len(jobs)

    # Minimize both measures
    fitness = 1 / (1 + makespan + mean_tardiness)

    return fitness, makespan, mean_tardiness

# ----------------------------------------
# 4. FITNESS-BASED PARENT SELECTION
# ----------------------------------------

def selection(population, fitness_values):

    ranked = sorted(
        zip(population, fitness_values),
        key=lambda x: x[1]
```

```
    )

    weights = list(range(1, len(ranked) + 1))
    total = sum(weights)

    point = random.uniform(0, total)
    cumulative = 0

    for i, weight in enumerate(weights):
        cumulative += weight

        if cumulative >= point:
            return ranked[i][0][:]

    return ranked[-1][0][:]

# ----------------------------------------
# 5. TWO-POINT CROSSOVER
# ----------------------------------------

def crossover(parent1, parent2):

    if random.random() < 0.8:

        p1, p2 = sorted(random.sample(range(len(parent1)), 2))

        child1 = parent1[:p1] + parent2[p1:p2] + parent1[p2:]
```

```
        child2 = parent2[:p1] + parent1[p1:p2] + parent2[p2:]

        return repair(child1), repair(child2)

    return parent1[:], parent2[:]

# ----------------------------------------
# 6. MUTATION AND REPAIR
# ----------------------------------------

def repair(chromosome):

    for i, part in enumerate(part_types):

        maximum = len(data[part]["plans"])

        if chromosome[i] < 1 or chromosome[i] > maximum:
            chromosome[i] = random.randint(1, maximum)

    return chromosome

def mutation(chromosome):

    if random.random() < 0.2:

        p1, p2 = random.sample(range(len(chromosome)), 2)
```

```
            chromosome[p1], chromosome[p2] = (
                chromosome[p2], chromosome[p1]
            )

            chromosome = repair(chromosome)

        return chromosome

# ----------------------------------------
# 7. GENETIC ALGORITHM
# ----------------------------------------

def genetic_algorithm():

    random.seed(1)

    population_size = 30
    generations = 90

    population = [
        create_chromosome()
        for _ in range(population_size)
    ]

    best_chromosome = None
    best_fitness = -1
```

```
stagnant = 0

for generation in range(generations):

    results = [evaluate(c) for c in population]
    fitness_values = [r[0] for r in results]

    best_index = fitness_values.index(max(fitness_values))

    current_best = fitness_values[best_index]

    if current_best > best_fitness:

        best_fitness = current_best
        best_chromosome = population[best_index][:]
        stagnant = 0

    else:
        stagnant += 1

    if generation % 10 == 0:
        print("Generation:", generation,
              "Best Fitness:", round(best_fitness, 8))

    # Elitism: preserve the best chromosome
    new_population = [best_chromosome[:]]
```

```
# Generate offspring
while len(new_population) < population_size:

    parent1 = selection(population, fitness_values)
    parent2 = selection(population, fitness_values)

    child1, child2 = crossover(parent1, parent2)

    child1 = mutation(child1)
    child2 = mutation(child2)

    new_population.append(child1)

    if len(new_population) < population_size:
        new_population.append(child2)

population = new_population

# Restart mechanism after 15 stagnant generations
if stagnant >= 15:

    print("Restarting population...")

    population = [best_chromosome[:]]

    # Generate some new random chromosomes
    while len(population) < population_size:
```

```
                population.append(create_chromosome())

            stagnant = 0

    return best_chromosome

# ----------------------------------------
# 8. DISPLAY FINAL RESULT
# ----------------------------------------

best = genetic_algorithm()

fitness, makespan, tardiness = evaluate(best)

print("\n========== FINAL RESULT ==========")

print("Best Chromosome:", best)

print("\nSelected Process Plans:")

for i, part in enumerate(part_types):
    print(part, "-> Process Plan", best[i])

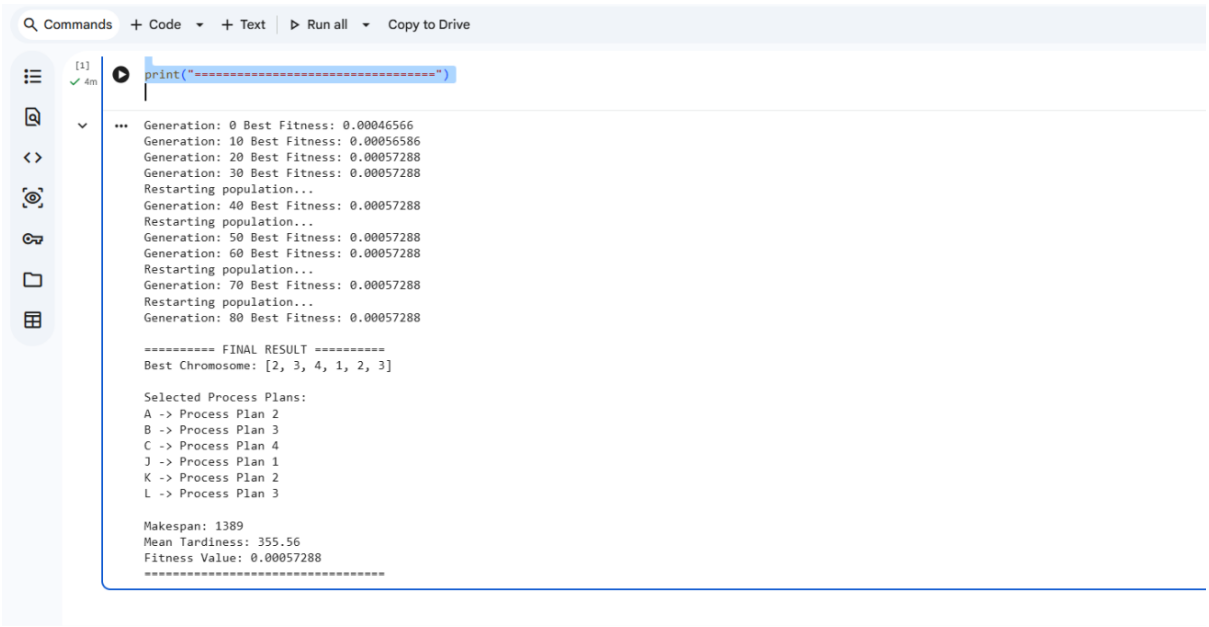
print("\nMakespan:", makespan)

print("Mean Tardiness:", round(tardiness, 2))
```

```
print("Fitness Value:", round(fitness, 8))

print("====================================")
```

output :



nischal ar

1WN24CS182